

```

1 import static org.junit.Assert.assertEquals;
2
3 /**
4  * @author Eric Zhou
5  */
6
7 public class CryptoUtilitiesTest {
8
9     /**
10      * Tests of reduceToGCD
11      */
12
13     /**
14      * testing ReduceToGCD with n is 0, m is 0, boundary case for m
15      */
16     @Test
17     public void testReduceToGCD_0_0() {
18         NaturalNumber n = new NaturalNumber2(0);
19         NaturalNumber nExpected = new NaturalNumber2(0);
20         NaturalNumber m = new NaturalNumber2(0);
21         NaturalNumber mExpected = new NaturalNumber2(0);
22         CryptoUtilities.reduceToGCD(n, m);
23         assertEquals(nExpected, n);
24         assertEquals(mExpected, m);
25     }
26
27     /**
28      * testing ReduceToGCD with n is 30, m is 21, routine test
29      */
30     @Test
31     public void testReduceToGCD_30_21() {
32         NaturalNumber n = new NaturalNumber2(30);
33         NaturalNumber nExpected = new NaturalNumber2(3);
34         NaturalNumber m = new NaturalNumber2(21);
35         NaturalNumber mExpected = new NaturalNumber2(0);
36         CryptoUtilities.reduceToGCD(n, m);
37         assertEquals(nExpected, n);
38         assertEquals(mExpected, m);
39     }
40
41     /**
42      * testing ReduceToGCD with n is 56, m is 98, routine test
43      */
44     @Test
45     public void testReduceToGCD_56_98() {
46         NaturalNumber n = new NaturalNumber2(56);
47         NaturalNumber nExpected = new NaturalNumber2(14);
48         NaturalNumber m = new NaturalNumber2(98);
49         NaturalNumber mExpected = new NaturalNumber2(0);
50         CryptoUtilities.reduceToGCD(n, m);
51         assertEquals(nExpected, n);
52         assertEquals(mExpected, m);
53     }
54
55     /**
56      * testing ReduceToGCD with n is 1, m is 999, large number test
57      */
58     @Test
59     public void testReduceToGCD_1_999() {
60         NaturalNumber n = new NaturalNumber2(1);
61         NaturalNumber nExpected = new NaturalNumber2(1);
62         NaturalNumber m = new NaturalNumber2(999);
63         NaturalNumber mExpected = new NaturalNumber2(0);
64         CryptoUtilities.reduceToGCD(n, m);
65         assertEquals(nExpected, n);
66     }
67 }

```

```

71     assertEquals(mExpected, m);
72 }
73
74 /*
75  * Tests of isEven
76  */
77
78 /*
79  * testing isEven with n is 0, boundary case for n
80  */
81 @Test
82 public void testIsEven_0() {
83     NaturalNumber n = new NaturalNumber2(0);
84     NaturalNumber nExpected = new NaturalNumber2(0);
85     boolean result = CryptoUtilities.isEven(n);
86     assertEquals(nExpected, n);
87     assertEquals(true, result);
88 }
89
90 /*
91  * testing isEven with n is 1, boundary case for n
92  */
93 @Test
94 public void testIsEven_1() {
95     NaturalNumber n = new NaturalNumber2(1);
96     NaturalNumber nExpected = new NaturalNumber2(1);
97     boolean result = CryptoUtilities.isEven(n);
98     assertEquals(nExpected, n);
99     assertEquals(false, result);
100 }
101
102 /*
103  * testing isEven with n is 666666, routine test
104  */
105 @Test
106 public void testIsEven_666666() {
107     NaturalNumber n = new NaturalNumber2(666666);
108     NaturalNumber nExpected = new NaturalNumber2(666666);
109     boolean result = CryptoUtilities.isEven(n);
110     assertEquals(nExpected, n);
111     assertEquals(true, result);
112 }
113
114 /*
115  * testing isEven with n is 1234567, large number test
116  */
117 @Test
118 public void testIsEven_1234567() {
119     NaturalNumber n = new NaturalNumber2(1234567);
120     NaturalNumber nExpected = new NaturalNumber2(1234567);
121     boolean result = CryptoUtilities.isEven(n);
122     assertEquals(nExpected, n);
123     assertEquals(false, result);
124 }
125
126 /*
127  * Tests of powerMod
128  */
129
130 /*
131  * testing powerMod with n is 0, p is 0, m is 2, boundary test
132  */
133 @Test
134 public void testPowerMod_0_0_2() {
135     NaturalNumber n = new NaturalNumber2(0);

```

```

136     NaturalNumber nExpected = new NaturalNumber2(1);
137     NaturalNumber p = new NaturalNumber2(0);
138     NaturalNumber pExpected = new NaturalNumber2(0);
139     NaturalNumber m = new NaturalNumber2(2);
140     NaturalNumber mExpected = new NaturalNumber2(2);
141     CryptoUtilities.powerMod(n, p, m);
142     assertEquals(nExpected, n);
143     assertEquals(pExpected, p);
144     assertEquals(mExpected, m);
145 }
146
147 /*
148  * testing powerMod with n is 17, p is 18, m is 19, large number test
149  */
150 @Test
151 public void testPowerMod_17_18_19() {
152     NaturalNumber n = new NaturalNumber2(17);
153     NaturalNumber nExpected = new NaturalNumber2(1);
154     NaturalNumber p = new NaturalNumber2(18);
155     NaturalNumber pExpected = new NaturalNumber2(18);
156     NaturalNumber m = new NaturalNumber2(19);
157     NaturalNumber mExpected = new NaturalNumber2(19);
158     CryptoUtilities.powerMod(n, p, m);
159     assertEquals(nExpected, n);
160     assertEquals(pExpected, p);
161     assertEquals(mExpected, m);
162 }
163
164 /*
165  * testing powerMod with n is 2, p is 3, m is 5, routine test
166  */
167 @Test
168 public void testPowerMod_2_3_5() {
169     NaturalNumber n = new NaturalNumber2(2);
170     NaturalNumber nExpected = new NaturalNumber2(3);
171     NaturalNumber p = new NaturalNumber2(3);
172     NaturalNumber pExpected = new NaturalNumber2(3);
173     NaturalNumber m = new NaturalNumber2(5);
174     NaturalNumber mExpected = new NaturalNumber2(5);
175     CryptoUtilities.powerMod(n, p, m);
176     assertEquals(nExpected, n);
177     assertEquals(pExpected, p);
178     assertEquals(mExpected, m);
179 }
180
181 /*
182  * Test for isWitnessToCompositeness
183  */
184
185 /*
186  * testing IsWitnessToCompositeness with n is 7, w is 3, routine test
187  */
188 @Test
189 public void testIsWitnessToCompositeness_3_7() {
190     NaturalNumber n = new NaturalNumber2(7);
191     NaturalNumber nExpected = new NaturalNumber2(7);
192     NaturalNumber w = new NaturalNumber2(3);
193     NaturalNumber wExpected = new NaturalNumber2(3);
194     boolean isWitness = CryptoUtilities.isWitnessToCompositeness(w, n);
195     assertEquals(nExpected, n);
196     assertEquals(wExpected, w);
197     assertEquals(false, isWitness);
198 }
199
200 /*

```

```

201  * testing IsWitnessToCompositeness with n is 9, w is 2, routine test
202  */
203  @Test
204  public void testIsWitnessToCompositeness_2_9() {
205      NaturalNumber n = new NaturalNumber2(9);
206      NaturalNumber nExpected = new NaturalNumber2(9);
207      NaturalNumber w = new NaturalNumber2(2);
208      NaturalNumber wExpected = new NaturalNumber2(2);
209      boolean isWitness = CryptoUtilities.isWitnessToCompositeness(w, n);
210      assertEquals(nExpected, n);
211      assertEquals(wExpected, w);
212      assertEquals(true, isWitness);
213  }
214
215  /*
216   * testing IsWitnessToCompositeness with n is 101, w is 2, large number test
217   */
218  @Test
219  public void testIsWitnessToCompositeness_2_101() {
220      NaturalNumber n = new NaturalNumber2(101);
221      NaturalNumber nExpected = new NaturalNumber2(101);
222      NaturalNumber w = new NaturalNumber2(2);
223      NaturalNumber wExpected = new NaturalNumber2(2);
224      boolean isWitness = CryptoUtilities.isWitnessToCompositeness(w, n);
225      assertEquals(nExpected, n);
226      assertEquals(wExpected, w);
227      assertEquals(false, isWitness);
228  }
229
230  /*
231   * Tests of isPrime2
232   */
233
234  /*
235   * testing IsPrime2 with n is 2, boundary test
236   */
237  @Test
238  public void testIsPrime2_2() {
239      NaturalNumber n = new NaturalNumber2(2);
240      NaturalNumber nExpected = new NaturalNumber2(2);
241      boolean isPrime = CryptoUtilities.isPrime2(n);
242      assertEquals(nExpected, n);
243      assertEquals(true, isPrime);
244  }
245
246  /*
247   * testing IsPrime2 with n is 4, boundary test
248   */
249  @Test
250  public void testIsPrime2_4() {
251      NaturalNumber n = new NaturalNumber2(4);
252      NaturalNumber nExpected = new NaturalNumber2(4);
253      boolean isPrime = CryptoUtilities.isPrime2(n);
254      assertEquals(nExpected, n);
255      assertEquals(false, isPrime);
256  }
257
258  /*
259   * testing IsPrime2 with n is 97, routine test
260   */
261  @Test
262  public void testIsPrime2_97() {
263      NaturalNumber n = new NaturalNumber2(97);
264      NaturalNumber nExpected = new NaturalNumber2(97);
265      boolean isPrime = CryptoUtilities.isPrime2(n);

```

```

266     assertEquals(nExpected, n);
267     assertEquals(true, isPrime);
268 }
269
270 /*
271  * testing IsPrime2 with n is 4, large number test
272  */
273 @Test
274 public void testIsPrime2_99999() {
275     NaturalNumber n = new NaturalNumber2(99999);
276     NaturalNumber nExpected = new NaturalNumber2(99999);
277     boolean isPrime = CryptoUtilities.isPrime2(n);
278     assertEquals(nExpected, n);
279     assertEquals(false, isPrime);
280 }
281
282 /*
283  * Tests of generateNextLikelyPrime
284  */
285
286 /*
287  * testing GenerateNextLikelyPrime with n is 4, boundary test
288  */
289 @Test
290 public void testGenerateNextLikelyPrime_4() {
291     NaturalNumber n = new NaturalNumber2(4);
292     NaturalNumber nExpected = new NaturalNumber2(5);
293     CryptoUtilities.generateNextLikelyPrime(n);
294     boolean isPrime = CryptoUtilities.isPrime2(n);
295     assertEquals(nExpected, n);
296     assertEquals(true, isPrime);
297 }
298
299 /*
300  * testing GenerateNextLikelyPrime with n is 99, routine test
301  */
302 @Test
303 public void testGenerateNextLikelyPrime_99() {
304     NaturalNumber n = new NaturalNumber2(99);
305     NaturalNumber nExpected = new NaturalNumber2(101);
306     CryptoUtilities.generateNextLikelyPrime(n);
307     boolean isPrime = CryptoUtilities.isPrime2(n);
308     assertEquals(nExpected, n);
309     assertEquals(true, isPrime);
310 }
311
312 /*
313  * testing GenerateNextLikelyPrime with n is 99999, large number test
314  */
315 @Test
316 public void testGenerateNextLikelyPrime_99999() {
317     NaturalNumber n = new NaturalNumber2(99999);
318     NaturalNumber nExpected = new NaturalNumber2(100003);
319     CryptoUtilities.generateNextLikelyPrime(n);
320     boolean isPrime = CryptoUtilities.isPrime2(n);
321     assertEquals(nExpected, n);
322     assertEquals(true, isPrime);
323 }
324
325 }
326

```